Project Design Document

ServiceNow Micro-Project: Implement Client Script & UI Policy (Incident)

2. Technical Architecture Design Overview

The engineering phase translates the system specifications into modular client-side configuration engines. To maximize browser optimization and prevent network lag, processing loops execute directly on the browser runtime environment. The systemic blueprint maps logical controls across two principal ServiceNow execution mechanisms: declarative validation rules (UI Policies) and procedural macro triggers (Client Scripts).

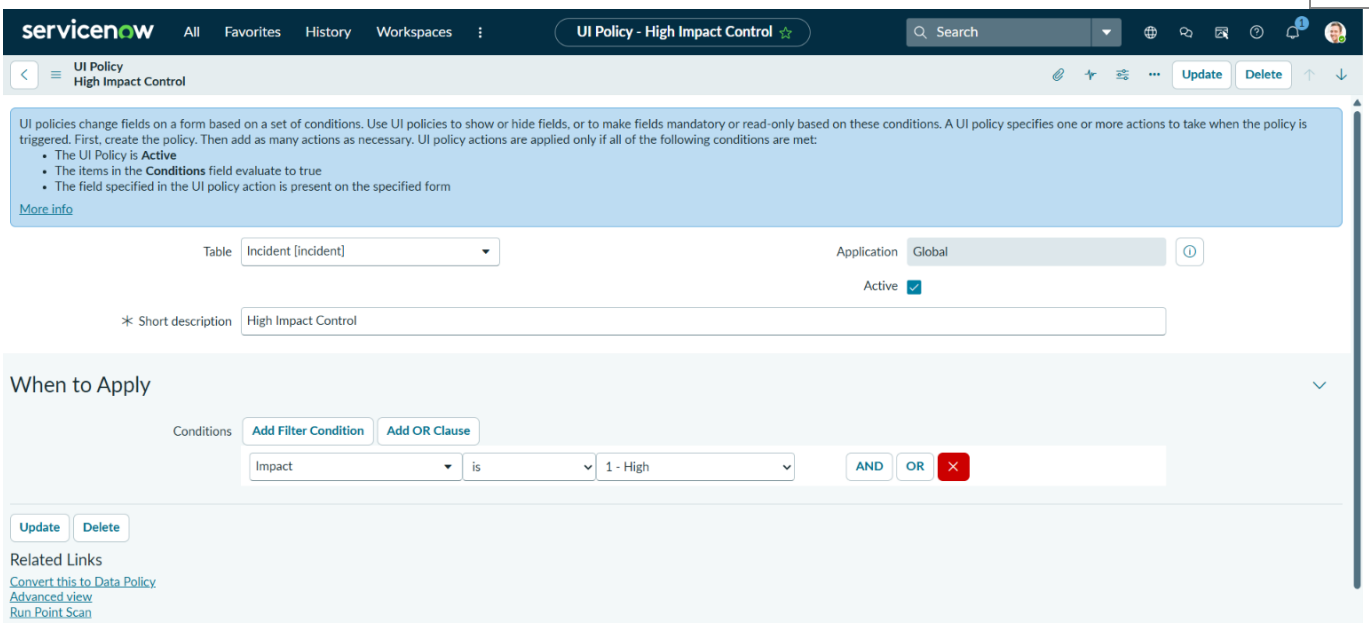
3. Programmatic Architecture Mapping

Policy / Script Identifier	Trigger Mechanism	Execution Parameters & Scope
UI Policy: High Impact Control	Form Load / Field Change (Impact == '1 - High')	Applies global declarative constraints. Makes Assignment Group mandatory and sets Urgency to read-only.
Client Script: Auto set urgency for high impact	onChange() (Field: Impact)	Intercepts data modification. Evaluates if structural index value equals '1', then sets 'urgency' to '1' and injects an info box.
Client Script: Prevent save if Assigned To missing	onSubmit()	Halts submission pipeline if 'assigned_to' is null while 'impact' matches High ('1'). Forces inline error box rendering.
Client Script: Prevent state change via list edit	onCellEdit() (Field: State)	Constructs grid row protection. Intercepts list editing actions on 'state' fields, alerts the client, and returns a false completion callback.

4. Detailed Blueprint Implementations

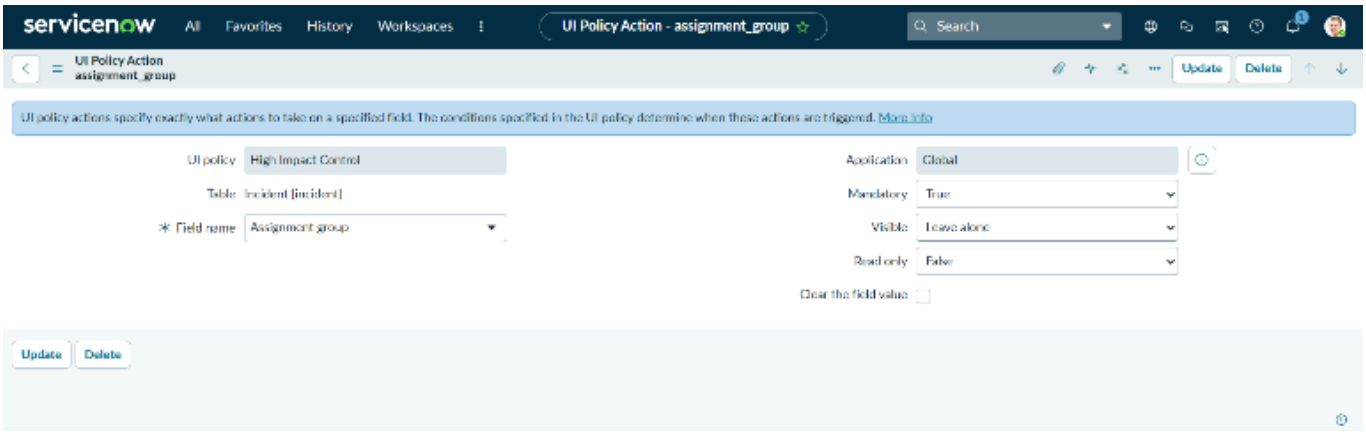
1.1 UI Policy Configuration Blueprint

- **Target Context Table:** Incident [incident]
- **Policy Core Settings:** Active: true | Global: true | Inherit: false
- **Reversal Behavior:** Reverse if false set to **true**. Reverts altered states instantly if fields fall below High criteria. • **Execution Conditions:** [Impact] [is] [1 - High]

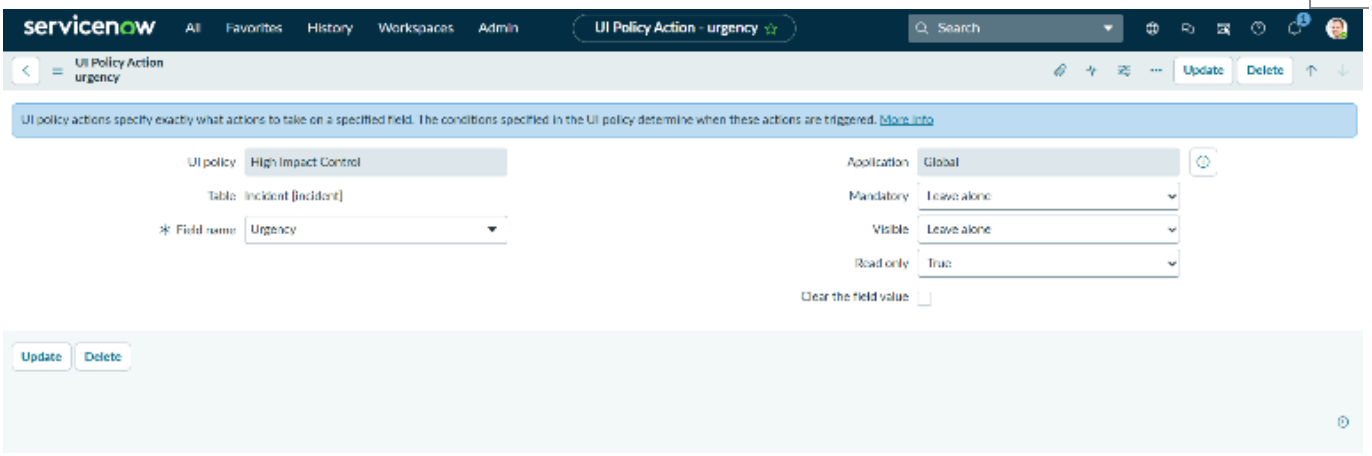


1.1 Target UI Policy Action Schemes

- **Action 1 (Assignment Group):** Mandatory: True | Visible: Leave Alone | Read-Only: Leave Alone



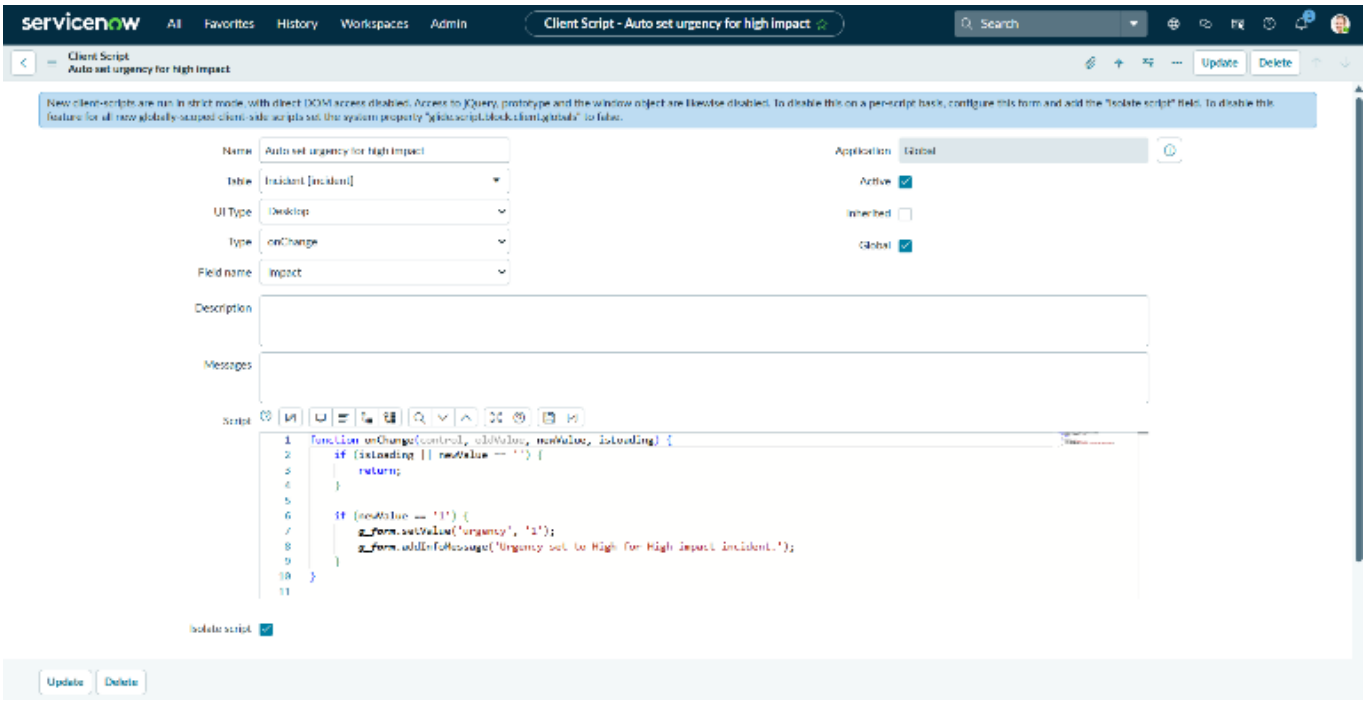
- **Action 2 (Urgency):** Mandatory: Leave Alone | Visible: Leave Alone | Read-Only: True



1. JavaScript Operational Subsystem Blocks

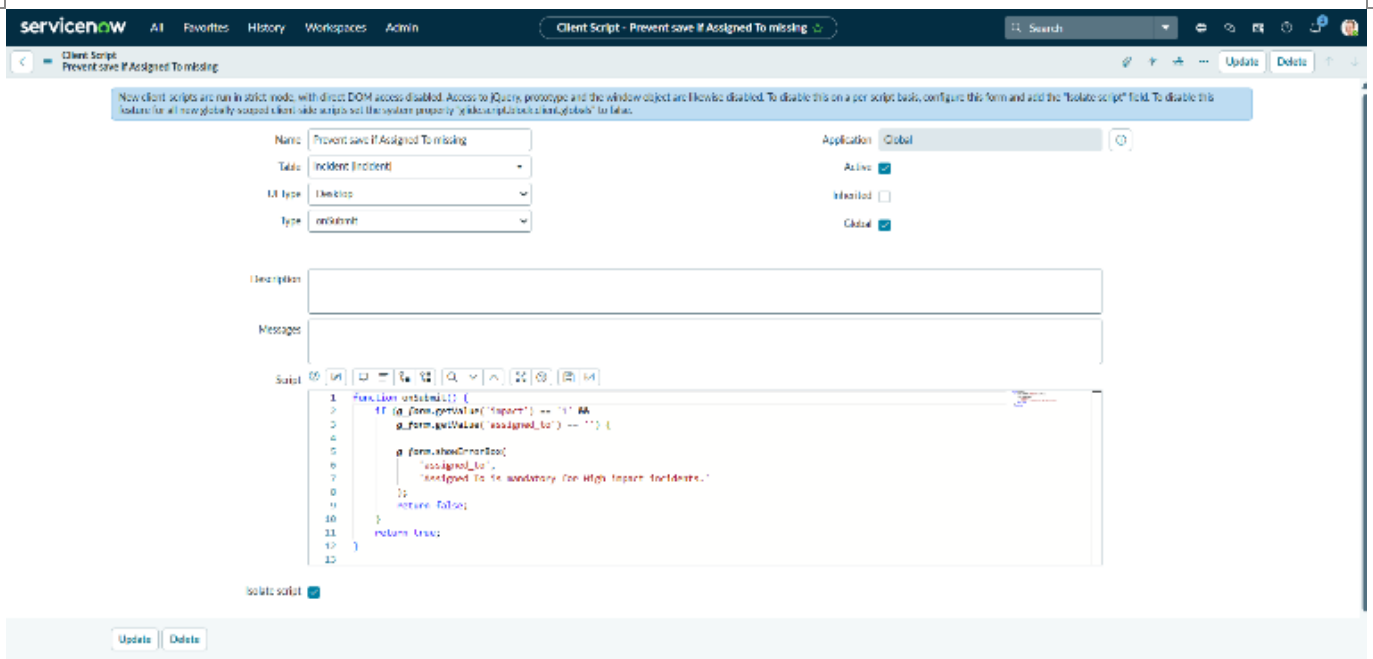
1.1 onChange Logic Module (Automated State Matrix Execution)

```
function onChange(control, oldValue, newValue, isLoading) {    if (isLoading || newValue == '') {        return;    }    if (newValue == '1') {        g_form.setValue('urgency', '1');        g_form.addInfoMessage('Urgency set to High for High impact incident.');    }
}
```



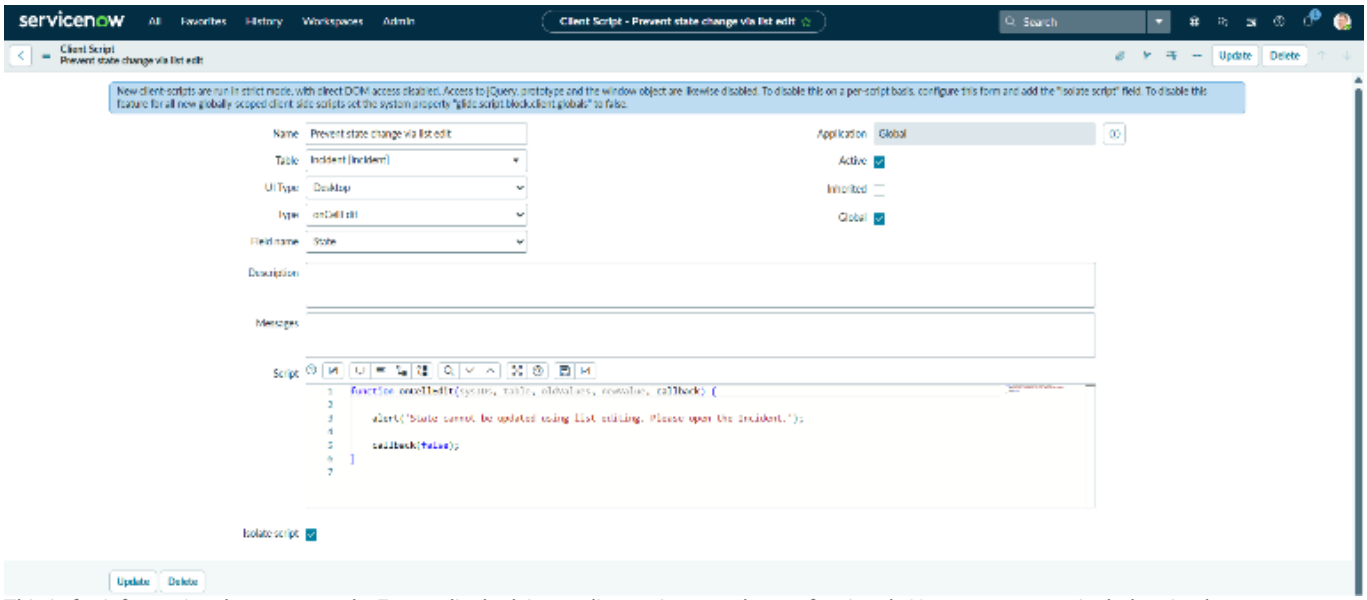
4.2 onSubmit Logic Module (Runtime Pre-Save Data Guard)

```
function onSubmit() {    if (g_form.getValue('impact') == '1' && g_form.getValue('assigned_to') == '')
{        g_form.showErrorBox('assigned_to', 'Assigned To is mandatory for High impact incidents.');        return false;    }    return true; }
```



4.3 onCellEdit Logic Module (Grid Modification Prevention Interceptor)

function onCellEdit(sysIDs, table, oldValues, newValue, callback) { alert('State cannot be updated using list editing. Please open the Incident.'); callback(false); }



This is for informational purposes only. For medical advice or diagnosis, consult a professional. AI responses may include mistakes.